REPORT

Workshop 1- Part 1

Braking Distance

María Ángeles Muñoz Juan-Dalac

Data Science and Engineering

Marta González Pérez

06.10.2025

Javier Fernández Ramos

ABSTRACT

This project explores the ability of neural networks to learn and reproduce real-world scenarios using synthetic generated data. We designed three candidate datasets based on the UV index, compound interest and braking distance, finally selecting the latter due to mathematical tractability. A dataset was generated from the braking distance equation and used to train a multilayer perceptron with SiLU activations. The optimized and tested 32-8 neuron architecture achieved exceptional performance (R2 = 0.99, MAE = 0.92) and showed an incredible alignment with the analytical solution, confirming its ability to approximate this physics non-linear problem. This project successfully demonstrated that a lightweight neural network can replicate the braking distance with remarkable accuracy, highlighting the potential of machine learning as a reliable and efficient tool for modelling actual safety and applied systems.

1. INTRODUCTION....4
2. OBJECTIVES.... 6
3. MATERIALS AND METHODS.... 7
3.1. Materials:....7
3.2. Methods:....7
4. RESULTS....10
5. CONCLUSION....14
6. REFERENCES AND BIBLIOGRAPHY....15

1. INTRODUCTION

This report consists of the design and implementation of a synthetic dataset based on a real-world mathematical model. This mathematical model has been chosen between three different options, the UV Index, the compound interest calculation and the braking distance. Among these, two datasets were developed as preliminary exploratory cases designed, while the third one was selected to be processed and trained to make predictions.

Starting with the UV Index numerical model, taking into account that the calculation of the UV Index is based on measuring the amount of UV radiation that reaches the Earth's surface. To calculate the UV Index, it is necessary to take several factors into account, such as geographic location, time of day, season, and atmospheric conditions. That is why mathematical models are used that combine data from the UV radiometer with information about these factors to calculate the intensity of UV radiation on a scale from 0 to 11+. The mathematical model used in this study is the following:

```
UVR(t=LSN) = UVW{TO(t=LSN),SZA(t=LSN)} * f(D) * f(A) * f(H)

UVI = UVR(t=LSN) / 0.025
```

As it can be observed, the main variables considered are, the solar zenith angle (θ), which is the angle between the local zenith and the line of sight from that point to the sun. The total ozone (TO), whose total column amount is dominated by the high concentrations of ozone in the stratospheric ozone layer. Finally, the altitude (h), the distance measurement between a reference datum and a point or object, is included through the altitude correction factor (f(H)). All three components, combined with a series of restrictions and correction factors, result in the formula shown before, which has led us to the design of the synthetic dataset via Python.

Later on, the decision to focus on compound interest was motivated by its well defined mathematical structure, its relevance in financial forecasting and its non-linear behaviour, which provided an excellent benchmark for evaluating the ability of the neural network to learn complex functional relationships from data alone. The compound interest formula offers a deterministic equation with clear input-output relationships:

$$A = P\left(1 + \frac{r}{n}\right)^{nt}$$

Where A is the final accumulated amount or the future value, calculated from P, the initial investment with an annual interest rate, r, in a number of years, t, within a number of times interest is compounded per year, n. Accordingly, we generated a synthetic dataset by sampling realistic ranges of the different inputs of the formula.

Continuing with the design of the braking distance model, it is important to understand that the total distance required for a vehicle to stop is not determined solely by the braking system itself, but by a combination of human and physical factors. To calculate this distance, two main components must be

taken into account: the driver's reaction time and the physical process of deceleration due to tire-pavement friction.

That is why mathematical models are used which combine both the behavioral aspect of the driver and the physical limitations of friction. The mathematical model used in this study is the following:

$$D_{total} = vt_{p-r} + \frac{v^2}{2\mu g}$$

The first variable is the initial velocity (v), which directly influences both components of the formula: the faster the vehicle moves, the longer the distance covered before and during braking. The second component is the driver's reaction time (t), representing the delay between perceiving a hazard and applying the brakes; during this interval, the vehicle continues to move at constant speed. The third element is the coefficient of friction (μ), which reflects the grip between the tires and the pavement. This coefficient varies significantly depending on road conditions, higher values on dry asphalt and much lower on wet or icy surfaces. Finally, gravity ($g = 9.81$ m/s²) plays a role as the universal constant that anchors the calculation of deceleration.

This approach allows us to model real driving conditions, since it integrates both the human response and the physical capacity of the vehicle-road interaction.

Starting with the design of the dataset for model development, it is necessary to consider the type of mathematical expressions that will be used during preprocessing. When building a neural network such as an MLP, the choice of formulas influences how easily the input data can be structured, normalized and processed.

In this context, the three possible formulas were considered and as can be observed, the first two expressions involve more complex operations, such as exponentiation with multiple parameters, trigonometric functions and power laws. These elements, while accurate in their respective domains, increase the computational and preprocessing complexity when adapting the formula to machine learning pipelines.

The third expression, corresponding to the UV Index calculated using the simplified parametrization, was selected precisely because of its simplicity. It combines only basic arithmetic operations (multiplication) with simple trigonometric evaluation, which are easier to preprocess and encode as features for the neural network. By reducing mathematical complexity at this stage, the dataset becomes more manageable, making the MLP model development process more efficient and straightforward.

2. OBJECTIVES

This project began by evaluating three real-world mathematical models — the UV Index, compound interest, and total braking distance — as candidates for neural network prediction. While all offered valuable learning opportunities, the braking distance formula was selected for its simplicity and applicability.

The primary objective is to design and train a neural network capable of accurately approximating the total braking distance using synthetic data generated from the physics-based equation. Specific goals to achieve our mission include:

- Generating a realistic dataset incorporating vehicle speed, tire–pavement friction, and driver reaction time using the physics equation.
- Preprocessing the data using standardization to ensure optimal neural network training and using SiLU activation functions.
- Experimenting with multiple MLP architectures (including 64–32, 32–16, and 32–8 neurons) to identify the most efficient and accurate configuration.
- Evaluating model performance using statistical metrics (MAE, R^2, MAPE) and visual diagnostics (scatter plots, residuals, training curves).
- Validating the final model against analytical calculations to confirm its fidelity to the underlying physical law.

Ultimately, this project aims to demonstrate how to implement a data-driven and machine learning solution to a real-world problem solving a non-linear equation accurately such as calculating the break distance.

3. MATERIALS AND METHODS

3.1. Materials:

For the development of the braking distance predictive model, several software libraries and tools were used. NumPy was employed for numerical operations and array management, allowing efficient manipulation of the generated datasets and random sampling for simulation purposes. Sklearn provided tools for data preprocessing and evaluation, including train_test_split for dividing the dataset into training, validation, and test sets, and StandardScaler for normalizing both input features and target variables. In addition, Sklearn metrics such as r2_score, mean_absolute_error, median_absolute_error, and mean_squared_error were used to quantitatively assess model performance.

Moreover, sktime.performance_metrics.forecasting provided additional regression metrics, including mean_absolute_percentage_error, mean_squared_percentage_error, and median_absolute_percentage_error, allowing evaluation of prediction errors in relative terms, useful when comparing predictions across different scales.

Matplotlib was used to visualize the data and the model's performance which allowed inspection of convergence, prediction accuracy, and potential overfitting.

Finally, for model construction and training, TensorFlow and Keras were used. The Functional API allowed building a multi-layer perceptron with flexible architecture, including input, hidden, and output layers. The Dense and Input layers defined the network structure, while the Adam optimizer was employed to update model weights efficiently during training.

Furthermore, it all has been computed on a MacBook Air M1 using Google Colab online environment.

3.2. Methods:

Starting with the design of the braking distance predictive framework, the code is structured to simulate, preprocess, and model vehicle stopping distances using machine learning. The overall purpose is to approximate the physical braking distance equation through a neural network (MLP) and compare the learned approximation with the analytical solution.

The first part of the code focuses on data generation. A function is defined to calculate the total braking distance based on the physical model. Random samples are generated for vehicle speed, tire-pavement friction coefficient, and driver reaction time, ensuring realistic values. The braking distance is then computed for each sample, and the resulting dataset is stored for further use.

The second stage involves data splitting, where the dataset is divided into training, validation, and test sets. This ensures that the model can be trained, tuned, and evaluated on separate partitions, preserving the integrity of performance assessment.

Initially, 20% of the data is separated to form the test set, which is kept aside to evaluate the model's performance at the final stage, while the remaining 80% becomes the combined training and validation set. From this 80%, a second split is performed: 80% of it is allocated to the training set and the remaining 20% to the validation set. This procedure results in a final distribution of 64% for training, 16% for validation, and 20% for testing.

Once the subsets are defined, the features are separated into x_train, x_val, and x_test, while the labels are assigned to y_train, y_val, and y_test. Finally, the shapes of each subset are printed, confirming that the splits have been executed correctly and providing information on the number of samples and features contained in each partition.

Next, data preprocessing is carried out. Both inputs (velocity, friction, reaction time) and outputs (braking distance) are normalized using StandardScaler, which works by centering the data around zero and scaling it to unit variance according to the formula:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation of the feature. This ensures that each variable has a mean of 0 and a standard deviation of 1, preventing features with larger scales from dominating the learning process. Normalization improves the convergence of gradient-based algorithms, such as linear regression, neural networks, or SVMs, and allows the model to learn more stable relationships, especially when the target variable has large or widely dispersed values. StandardScaler is preferred when the data is approximately normally distributed, as it both centers the data and standardizes its spread.

The fourth part is the design of the neural network. A functional Keras model is implemented, consisting of an input layer, two hidden layers with SiLU activations, and a final linear output layer. SiLU combines the properties of the sigmoid and identity functions: it is smooth and differentiable, which helps gradient-based optimization, and it is non-linear, allowing the network to learn complex patterns. Unlike ReLU, it does not completely zero out negative values but scales them smoothly, while positive values remain roughly linear. This feature often improves convergence and performance, as small negative gradients are preserved, helping the model learn more effectively. In short, SiLU "gates" the input by its own sigmoid, providing a balance between linearity and non-linearity that can enhance training and accuracy in deep networks.

The network takes three input variables: initial speed (m/s), tire-pavement friction coefficient, and driver reaction time (s). Based on these inputs, the model predicts the total braking distance (m). The first hidden layer has 32 neurons with a SiLU activation function, followed by a second hidden layer with 8 neurons, also using SiLU. The output layer consists of a single neuron with a linear activation to produce a continuous value representing the braking distance.

However, different pair configurations for the number of neurons in the hidden layers were tested, including 32 and 16, 64, and 32 and 32 and 8. These configurations were evaluated by observing the training and validation loss curves, as well as the scatter and residual plots of predicted versus true braking distances.

The 32 and 16, and the 64 and 32 configurations showed slightly higher training times and, in some cases, overfitting, as indicated by a larger gap between training and validation loss. In contrast, the 32 and 8 configuration provided a good balance, the model converged efficiently, training and validation losses were close, and the predicted distances closely matched the true values.

Based on these results, 32 and 8 neurons were chosen for the hidden layers, as this configuration offered stable learning, reduced complexity, and sufficient capacity to accurately approximate the braking distance equation without overfitting.

This design ensures that the model can learn the non-linear relationships between the inputs and the braking distance while remaining simple and interpretable. The function returns the compiled model ready for training.

Following model compilation, the code executes the training phase, where the neural network is trained for 70 epochs with batch size 32, using the scaled training and validation sets. This block first creates a neural network using functional_model with input_dim = 3, meaning each input has three features. The model is compiled with the Adam optimizer (learning rate 0.001), MSE as the loss function to penalize larger errors, and MAE as a metric to monitor average error. During training, MSE and MAE are computed for both sets, and the results are stored in history to track learning progress and detect potential overfitting.

In summary, this block builds, configures, and trains the network, allowing it to gradually improve predictions while monitoring performance on validation data.

After compiling and training the model, the evaluation phase is carried out. Several visualization functions are implemented:

- plot_training_curves that shows the evolution of training and validation loss, helping detect overfitting.

- plot_scatter that compares predicted versus true values and computes the R^2 score, evaluating model accuracy.

- plot_residuals and plot_error_histogram that analyze prediction errors, providing insight into error distribution and potential biases.

Finally, the code includes a sanity check. A specific set of parameters is evaluated both manually (using the physical braking equation) and through the trained neural network. The relative error between the exact solution and the ML prediction is calculated, verifying how well the network has approximated the true physical model.

In summary, the code integrates physics-based modeling with machine learning. It starts from the analytical braking distance formula, generates synthetic data, preprocesses it for neural network input, and develops an MLP capable of learning the mapping between inputs and braking distance. The combination of visual diagnostics and statistical metrics ensures that the learned model is both interpretable and comparable to the underlying physical law.

4. RESULTS

After training our model and testing it we obtained some metrics and results that give us the needed insights to prove that our model successfully learned the underlying relationships and has achieved its purpose.

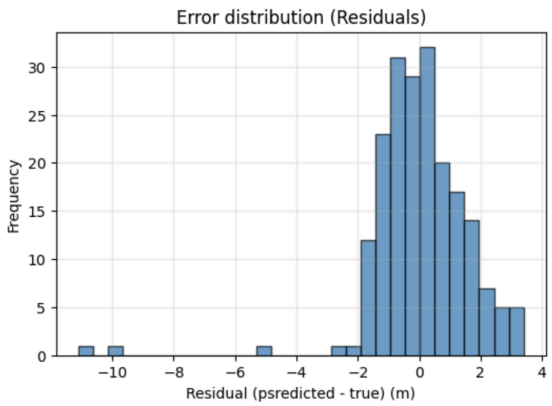
To check the performance of the neural network we employed a set of regression metrics and visual diagnostics. Each metric provide an insight into different aspects of the model accuracy, error distribution and practical reliability:

- Mean Absolute Error (MAE) measures the average magnitude of prediction errors in the same units as the target, in this case meters. It is robust to outliers and gives a direct look into how far are predictions from real values on average.
- Median Absolute Error (Median AE) complements MAE indicating the typical error experienced, useful for understanding the central tendency.
- Coefficient of determination (R2) quantifies the proportion of variance in the true output experienced, a value close to one indicates that the model captures the variability in the braking distance.
- Mean Absolute Percentage Error (MAPE) and Median Absolute Percentage Error (MedAPE) expresses errors as percentages of the true values enabling scale-independent interpretation, especially valuable when braking distances vary widely.

In addition to the numerical metrics, we used visual tools like the training curve, to diagnose convergence behaviour, detect bias, assess error and validate alignment with the analytical formula.

As previously discussed, we performed a series of experiments using different numbers of neurons in order to analyze how this parameter affects the overall performance of the model. In the first two experiments, we decided to present only the most relevant configurations, focusing on those that provided the most representative or insightful results.

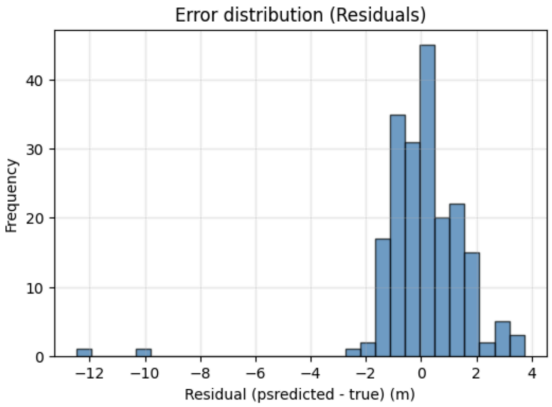
1. Configuration with 32 and 16 neurons.




```
Further evaluation:
{'MAE': 1.0880129543629145,
 'Median AE': 0.8122175783379557,
 'MSE': 2.718077075298942,
 'R2': 0.9986732576814341,
 'MAPE (%)': 8.785736356705396,
 'MSPE (%)': 10.437428770209808,
 'MedAPE (%)': 2.28693239714741}
```

The model shows solid performance but slightly high errors, with a MAE of 1.088 and MAPE of 8.79%, the model accurately approximates the braking distance but it exhibits large deviations. We can see in the residuals distribution that predictions are around +-2.5m of the actual value but the frequency at these extreme values is very high.

2. Configuration with 64 and 32 neurons.

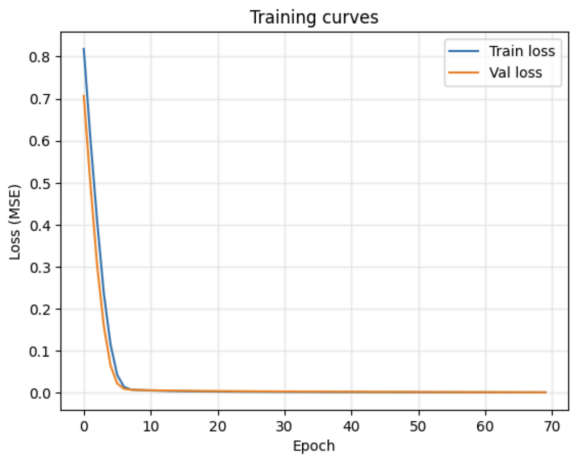



```
Further evaluation:
{'MAE': 1.0181395849000618,
 'Median AE': 0.7807718625585167,
 'MSE': 2.6490020931232836,
 'R2': 0.9986411507007852,
 'MAPE (%)': 12.993076127037211,
 'MSPE (%)': 43.361933908095374,
 'MedAPE (%)': 1.690218372258672}
```

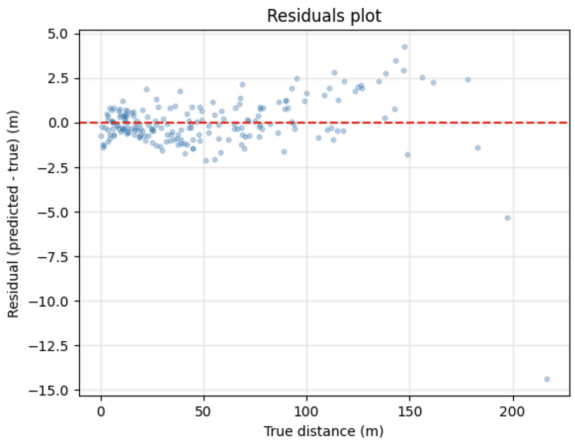
Now, when trying with the 64-32 neuron configuration demonstrated stronger learning capability but slightly high prediction errors, with metrics like MAPE being 12.99%. The model yet showed increased relative error although the error histogram shows a symmetric distribution around zero.

Now, we carry out a more detailed analysis of the graphs and results obtained for the models with 32 and 8 neurons, since it is our definitive model.

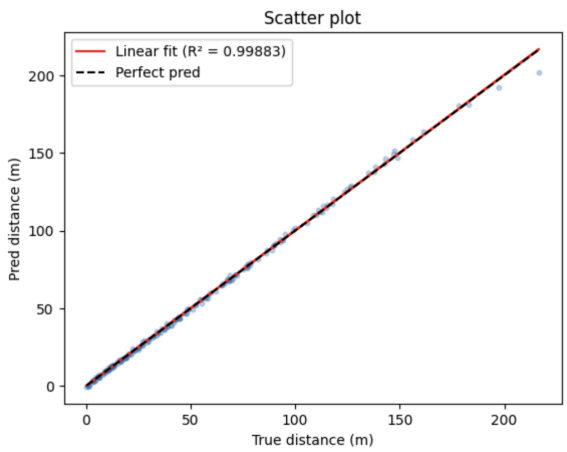
3. Final configuration with 32 and 8 neurons.



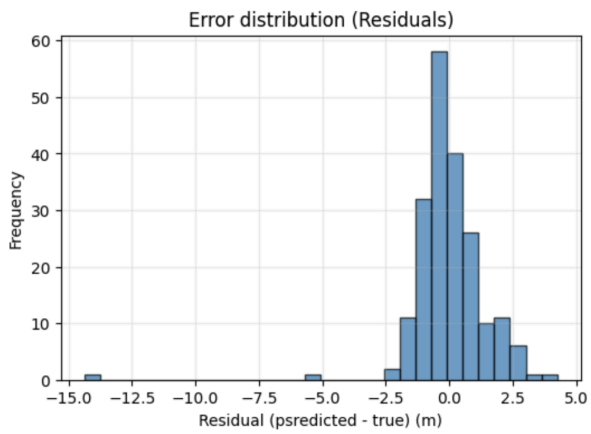
Starting with the training curves where we can see how rapid convergences, within the first 10 epochs of the model. Both training and validation loss stabilize near 0.01, indicating excellent learning without overfitting. The model continues to improve slightly through training in the remaining epochs. This said, the model generalizes well evidenced by the minimal gap between train and validation loss.



On the other hand, as we can observe in the plot the average error obtained (predicted - true value). Errors are centered around zero across the full range, confirming no bias. We can observe some minor deviated points at higher braking distances that reflect the limitations in learning non-linear relationships under the sampling made. This does not mean that the model fails, as it makes the model more robust to real-world deployment, where such variations are common.



Later, as we observe in the scatter plot where we compare predicted and true braking distances where the red line fit shows the R2 score obtained, of 0.9983 and the perfect prediction in back. This plot indicates accuracy as the points lie close to this perfect prediction.



The histogram shows a tight symmetric distribution centered at zero. Most errors fall within 2.5 meters plus or minus. Errors are small, unbiased and normally distributed, which is perfect.

Now we are focusing on the numerical methods where we can see that MAE is 0.92, this means that on average predictions are off by less than one meter from the true value, which is exceptional. Moreover, R2 is nearly one so the model captures almost all the variability in the braking distance. We obtained a MAPE and a MedAPE of 6.3% and 1.95% which means that the relative error remains low, outstanding robustness.

```
Further evaluation:
{'MAE': 0.9237315318632091,
 'Median AE': 0.6955441709291681,
 'MSE': 2.358316239417677,
 'R2': 0.9988276174801749,
 'MAPE (%)': 6.332490667131273,
 'MSPE (%)': 7.335843068450284,
 'MedAPE (%)': 1.9464900955120898}
```

Finally we did the sanity check to validate the model using a specific test case and we obtained a really small relative error, which proves that our model achieves its goal.

Although the model achieved satisfactory performance across all tested neuron counts, the configurations with 32 and 8 neurons stood out as offering slightly better accuracy and stability, while also reducing computational complexity and training effort, that is why we have chosen this number of neurons.

5. CONCLUSION

All in all, we implemented a neural network that can learn and accurately approximate a real-word physical and mathematical formula, the braking distance, using only synthetic data. By generating realistic samples and training a compact multilayer perceptron with a 32-8 neuron architecture, we achieved an exceptional predictive performance, as seen in the metrics of the results. The model not only captures the non-linear relationships between speed, friction, reaction time and stopping distance, but also generalizes well without overfitting.

The use of SiLU activation functions, feature standardization and careful hyperparameter tuning, checking different schemes and testing different neuron configurations proved our critical look into achieving this high fidelity model. Visual diagnostics such as training curves, scatter plots and residual analysis consistently validated the reliability of the model, showing minimal bias, rapid convergence and robustness across the different ranges of scenarios.

Furthermore, the sanity checks yielded a negligible relative error, confirming that the learned model faithfully replicates the original physical law. This bridges the existing gap between classical physics and modern machine learning, illustrating that data-driven solutions like this one can serve as fast and accurate solvers for analytical problems like predicting the brake distance. This approach could be of incredible help at implementing autonomous driving or advanced driver-assistance systems like ADAS.

While the current model performs like a snap, several enhancements could increase its applications to the real world like incorporating real-world data instead of synthetic datasets or having probability and statistics into account. However, this project validates that even a simple neural network can effectively emulate deterministic physical and actual problems. This project lays a basic foundation for future work at joining physics based machine learning and intelligent and secure transportation and automobilistic systems.

6. REFERENCES AND BIBLIOGRAPHY

- European Space Agency. (s. f.). *TEMIS — UV index and UV dose: Details of the calculation*. https://www.temis.nl/uvradiation/product/uvi-uvd.html

- Matematizame. (s. f.). *Fórmula del interés compuesto: descubre cómo calcularlo*. https://matematizame.com/formula-del-interes-compuesto-descubre-como-calcularlo/

- Wikipedia. (2024, 10 de septiembre). *Distancia de frenado*. En *Wikipedia, La enciclopedia libre*. https://es.wikipedia.org/wiki/Distancia_de_frenado

- Scikit-learn developers. (s. f.). *Model evaluation: quantifying the quality of predictions*. En *scikit-learn documentation*.https://scikit-learn.org/stable/modules/model_evaluation.html